

Coding for AI: Problem Set 5.2

Navneet Das *

September 7, 2026

Deliverable 2: Header Documentation

Why raw text must be normalised before counting:

Text normalisation is completely essential because Python evaluates strings with exact, case-sensitive equality. If we do not actively convert words to lowercase and strip bounding punctuation, words like "AI", "ai", and "AI," will be treated by the dictionary as three entirely different keys. Normalisation cleans the data so that semantic equivalents are properly counted together as a single word entity.

The difference between `set_a - set_b` and `set_a ^ set_b`:

The mathematical expression `set_a - set_b` performs a directional set difference; it returns only the elements that exist in `set_a` but are entirely absent from `set_b` (i.e., unique to Platform A). Conversely, the expression `set_a ^ set_b` represents the symmetric difference. It isolates all elements that are unique to exactly one of the sets, effectively combining the uniquely occurring elements from both `set_a` and `set_b` into a single set.

Deliverable 1: Source Code

The following is the combined implementation code for `social_media_auditor.py`. A docstring has been added to fulfil the header comment requirement specified in the assignment block.

```
1 """
2 Author: [Navneet DAS]
3 Course: Coding for AI
4 Assignment: Problem Set 5.2 - The AI Social Media Auditor
5
6 Description:
7 This script performs text normalisation to accurately count word
8 frequencies
9 (Task 1) and utilizes set operations to deduplicate and compare
10 follower lists
11 across two different social media platforms (Task 2).
12 """
13 import string
14
15 text = """AI is changing the world. ai is powering new tools, and AI
```

*Roll No: [17]

```

14 is helping researchers learn faster. The world loves AI, and the world
    depends on it more each year."""
15
16 word_count = {}
17 for raw_words in text.split():
18     # Convert to lowercase and strip all punctuation
19     cleaned_word = raw_words.lower().strip(string.punctuation)
20     if not cleaned_word:
21         continue
22     word_count.setdefault(cleaned_word, 0)
23     word_count[cleaned_word] += 1
24
25 print("--- Task 1: Word Frequency Count ---")
26 print(word_count)
27 print("\n")
28
29 platform_a_followers = ["u101", "u102", "u103", "u104", "u105"]
30 platform_b_followers = ["u103", "u104", "u106", "u107"]
31
32 # Convert lists to sets
33 platform_a_followers = set(platform_a_followers)
34 platform_b_followers = set(platform_b_followers)
35
36 print("--- Task 2: Follower Deduplication ---")
37 print(f"Follower on both platforms (&) : {platform_a_followers &
    platform_b_followers}")
38 print(f"Unique to Platform A (-) : {platform_a_followers -
    platform_b_followers}")
39 print(f"Unique to Platform B (-) : {platform_b_followers -
    platform_a_followers}")
40 print(f"Unique to exactly one platform (^) : {platform_a_followers ^
    platform_b_followers}")

```

Listing 1: social_media_auditor.py

Program Output

When executed, the program processes the sample text and follower lists, producing the exact output required:

```

{'ai': 4, 'is': 3, 'changing': 1, 'the': 3, 'world': 3, 'powering': 1,
 'new': 1, 'tools': 1, 'and': 2, 'helping': 1, 'researchers': 1, '
 learn': 1, 'faster': 1, 'loves': 1, 'depends': 1, 'on': 1, 'it': 1,
 'more': 1, 'each': 1, 'year': 1}

Follower on both platforms (&) : {'u103', 'u104'}
Unique to Platform A (-) : {'u105', 'u101', 'u102'}
Unique to Platform B (-) : {'u107', 'u106'}
Unique to exactly one platform (^) : {'u107', 'u105', 'u102', 'u101', '
 u106'}

```

Note: Set display order may vary slightly on different machines as Python sets are unordered.

Deliverable 3: Submission Note

For my test paragraph in Task 1, my program correctly processed the unstructured string into discrete words. After standardising the text to lowercase and completely stripping all punctuation, the dictionary correctly grouped variations like 'AI', 'ai', and 'AI,' together. This resulted in an accurate final count of 4 for the single key 'ai', which matches my hand-worked expectations perfectly. For Task 2, using Python's native set operations on the provided follower lists successfully identified the overlapping followers ('u103' and 'u104'). Furthermore, the directional set differences effectively isolated the unique followers for Platform A ('u101', 'u102', 'u105') and Platform B ('u106', 'u107') independently. All program outputs generated precisely match the anticipated mathematical set calculations, confirming both data structures function exactly as intended.